Alloy

Sources

Alloy 6 - Practical Guide
03a.AlloyPart1-2.pdf (slides)

Alloy is a specification language, made to write documentations about software designs: a single alloy file is to be seen as a source code, runnable through tools. The aim of the language is to test specifications to be sure that a specification is well defined and written.

How does Alloy work

Being a tester, Alloy is based on the assumption to try to **generate corner cases** as frequently as possible: that's why every time we execute our code we could get a **different result**. Be warned that every assertion, test or case might be covered in one simulation and completely ignored in the next one. To generate a new set of results, it is possible to press the `new` button in the **Show** window.
To execute something, we have the `run` command. To generate something as an example, it is possible to write:

```
[...]
run example {}
```

Alloy works by defining the objects where we need to work, not by using default types such as `Int` or `String` (***that should always be avoided!***): even if it is counter intuitive, if a file has a name it will be of **type Name** (for example, having lots of instances of said type just to describe the actual name) and **not String**.

Signatures, Facts & Assertions

900_AlloyIntroduction

Signatures

Signatures are sets that group together some elements of a certain domain: they can generate atomic objects (top-level signatures) or specify a correlation between objects (subset signatures).

```
sig Object{}            // top-level signature, atomic
sig File in Object{}    // subset signature
sig Dir in Object{}
```

By running `run example{}`, a configuration of objects is shown: we can notice after some examples that this configuration can be seen.

image

Since we did not specify that a file cannot be a directory and vice-versa, the system accepts this configuration: it is our job to write the right constraints to prevent this.

Facts

We can introduce `facts`, **rules** that are never broken and that use objects previously declared:

```
fact {no File & Dir}    // "no" checks for set emptyness, "&" is the intersection
operator
```

But now we notice that also this configuration can appear:

image

We can so add a new fact inside our set of facts that prevent us to have objects that are neither Dir or File: multiple constraints inside a fact are modeled like if an `and` operator is in between them.

```
fact{
    no File & Dir
    Object = File + Dir // the Object set is composed of the union of the two
sets File and Dir
}
```

Facts can have names for readability

Checks & Assertions

We can also include "***called rules***", rules that are defined in our file but only valid once called: those are `checks`, and can be useful when testing specific corner cases and to show **counter examples**. Counter examples are configurations shown by Alloy that fail to satisfy the check, that's why it is useful to use checks on the run.

```
check {no File & Dir}   // in this case, no counter examples are shown (check is
ok)
```

Checks are mostly used in combination with **assertions** to use them when needed (if an assertion is defined somewhere with a name, we can call and check said assertion).

```
assert no_partitions{
    Object - Root = Root.^(entries.object)
}
[...]
check no_partitions
```

Extensions & Enumerations

Extensions

Normally, to specify that signatures are disjoint sets it is advisable to specify extension properties: when two objects are extensions of another object it becomes clear that those two objects are disjoint sets.

```
sig File extends Object{}
sig Dir extends Object{}
// File and Dir are completely disjoint

check {no File & Dir}   // the analyzer refuses to execute this command: wrong by definition
```

Note: extended objects are also easier to notice on graphs:

image

Enumerations

Sometimes we need to specify data types that extends abstract concepts: instead of declaring firstly an `abstract sig` and then `one sig` (one stands for "exactly one instance" of the atom) for every sub-signatures, it is good practice to use enumerations.

```
abstract sig Color {}
one sig Red, Green, Blue extends Color {}
\\becomes:
enum Color { Red, Green, Blue }
```

Order matters in enumerations; we will see how this can be useful in some cases

Scopes

The Alloy compiler by default assigns 3 as the maximum number of iterations of a top-level signature: this means that when we run our program, we can get from 1 to 3 iterations of said objects. We can change this setting with the command `for` for each signature that we want to (even to sub-signatures).

```
run example{} for 4 but 2 Object, exactly 3 File
```

If we modify the default scope of a sub-signature, to accomodate the new rule even the top-signature scope will be modified; if we try to force the scope of a signature to assume a value of iterations that is coherently wrong (we defined it as a `one sig`), we will get an error.

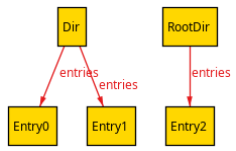
Field declaration

To declare complex signatures, we can use fields to add attributes-like entities with custom types: Alloy models them as **relations**.

> Classical Alloy motto: "Everything is a relation!"

Relations are defined inside the former object of the relation (if A → B, then relation is to be defined in the signature of A) and must have a name.

```
// a field called entries is defined as relation between Dir and Entry
sig Dir extends Object{
    entries: set Entry      // set is multiplicity (1 Dir -> lots of Entry
through entries)
}
```



image

There exists multiple adjectives to describe multiplicity:

- `set`: inside X, `rel: set Y` means for every X we have multiple Y
- `one`: inside X, `rel: one Y` means for every X we have exactly one Y
- `lone`: inside X, `rel: lone Y` means for every X we have zero or one Y
- `some`: inside X, `rel: some Y` means for every X we have at least one Y

The default is **one**.

Property Specification

Thanks to the graphical approach that Alloy gives us, it's easy to understand what are the problems of a badly written specification, but often it's not immediate to see them: the use of `facts` and `assertions` is essential to limit errors and malformed phrases to try to close up the problem and limit mistakes.
Facts are written using a sort of extension of *first-order logic* called **relational logic** (introduces relational operators): operators such as `&`, `+` and `-` are obviously present, with the adding of `.` (the **composition** operator), `~` (the **transpose** operator) and `^` (the **transitive closure** operator).

Composition

The composition operator allows us to navigate through fields of objects to obtain the relative atoms: it is key to fully understand how to correctly code with Alloy.

```
e.name      //with e as an instance of a singleton containing an Entry, we can
obtain its name
```

If we apply the binary operator to a set that is not a singleton, we obtain multiple entries: the result of the operator is always a set of values that can be limited to one, can be empty or can be made up of more values. Composition can be chained with other compositions to obtain indirectly connected atoms.

```
d.entries.object    //--> get entries from directory d --> get all objects from
said directories
```

Transpose

The transpose operator applies only to binary relations and takes the mirror image of said relation. Basically, gives back the inverse relation and it's useful to reverse orders or gain definitions from relations

```
p = {(a0, a1, a2), (b0, b1, b2)}
~p = {(a2 ,a1, a0), (b2, b1, b0)}

// if we want to specify the parent relation having only the children one:
parent = ~children
// and so on:
wife = ~husband
```

Transitive Closure

Transitive closure of an expression r gives us the power to take the **union of all possible compositions** of said expression r.

```
^r => r + r.r + r.r.r + [...]
```

This gives us a powerful tool to navigate through sets that are **interlinked** to each other and formed of the **same structure**, we can now formulate more generic assertions and facts that model general properties of our system. If we want, for example, to say that every object is reachable from the root, we can write the following assertion:

```
assertion no_partitions{
    Object - Root = Root.^(entries.object)
}
```

Thanks to the repetitiveness of the system that we have (filesystem example) the transitive closure is really powerful.

Other operators and keywords

Facts and assertions are built upon relational logic, the composition operator and logical operators (such as `implies`), with the adding of special keywords like `all`, `in` and `no`. `all` is used when declaring that the global set of a certain instance must satisfy the following condition, `in` is used when the following expression contains as a set the previous specified set, `no` is used to specify that the following expression has to contain an empty set.

```
// Entries cannot be shared between directories
fact{
    all x, y: Dir | x != y implies no (x.entries & y.entries)
}
```

Vadevecum on binary relations

Being Alloy a specification tester language, various predicates produce the same result and more than one solution is (almost) always possible: to model some of the properties of the relations that we need it is important to have a common ground, such as this.

injective	entire	simple	surjective
`A lone -> B`	`A -> some B`	`A -> lone B`	`A some -> B`
representation	function	abstraction	
`A lone -> some B`	`A -> one B`	`A some -> lone B`	
injection	surjection		
`A lone -> one B`	`A some -> one B`		
bijection			
`A one -> one B`			

image

So, for example, the latter fact becomes:

```
// Entries cannot be shared between directories
fact{
    entries in Dir lone -> Entry
}
```

> The relation is forced to be **injective**: no two atoms of the source signature point to the same target atoms.

Alloy 6

GitHub - lucapada/ResearchProjectAlloy6
With the introduction of a new version of Alloy, mutable structures are added to be used aside a simple concept of time: relations, entities and object are no longer defined as immutable atoms but as entities that can change.

Variables and Instants

Variables

A mutable relation is introduced with the keyword `var`: this enables the possibility to the relation to allow more instances connected to it (or also, less).

```
sig Name, Addr {}
sig Book {
    var addr: Name -> lone Addr
}
// addr can now be modified by other facts
```

To actually modify a `var` method, we need to introduce the concept of instants.

Instants

Instants are single execution steps: until now it's like we reasoned upon a single instant. The collection of those instances gives us multiple snapshots that show us the evolution of the system. We model instants with the keyword `after` and with the `'` and `''` operators.

Future and Past operators

Time operators can be summarized with the following table, coupled by their past and future versions.

FUTURE	PAST
`always`	`historically`

FUTURE	PAST
`eventually`	`once`
`after`	`before`
`until`	`since`
`releases`	`triggered`
`;`	

Always - Historically

Always and Historically model facts that are always true from a certain time in the past or in the future. `always P` means that predicate P **will always be true** from now on, `historically P` means that, until now, P **was always true**.

Eventually - Once

Eventually and Once model facts that are going to happen or have happened at least once. `eventually P` means that predicate P from now on **will happen at least once**, `once P` means that predicate P **happened at least one time**.

After - Before

After and Before model facts that are sure to happen exactly in the next or previous instant of time: `after P` means that predicate P **will happen in the next iteration**, `before P` means that predicate P **happened exactly the instant after the running one**.

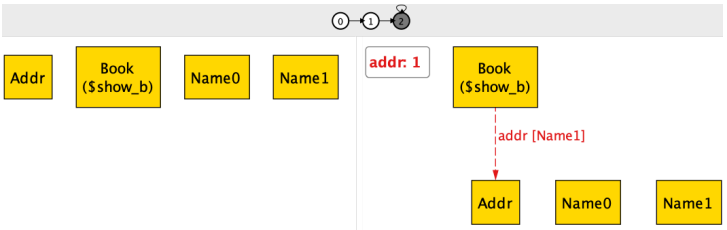
```
pred show [b: Book] {
    #b.addr = 0                 <-- first instant
    after (                     <-- second instant
        #b.addr = 1
        and
        after(                  <-- third instant
            #b.addr = 2
        )
    )
}
```

It is possible to model instants in a quick way with `;`:

```
pred show [b: Book]{
    #b.addr = 0; #b.addr = 1; #b.addr = 2
}
```

To run a finite number of instants, we can edit the `run` command with the `steps` keyword:

```
run show for 3 but 1 Book, 3 steps
```



image

Instances are shown in the upper section of the Alloy analyzer, it's easy to toggle the preferred instant and see the situation of all entities.

Triggered - Releases

These are binary operators: `P releases Q` means that once P is true, it is not mandatory that Q is true (normally, until P is said to be true Q must be true). On the other hand, `Q triggered P` means that P was at least true since Q was true.

' Operator

We can also predicate on the future instance of a relation by adding the `'` operator:

```
pred add [b: Book, n: Name, a: Addr]{
    b.addr' = b.addr + n -> a           <-- b.addr' refers to b.addr in next
instant
}
```

This convenient operator allows us to better model all frequently used predicates (that model, for example, **operations**): we saw the `add` predicate, we can also model the `del` predicate.

```
pred del [b: Book, n: Name]{
    b.addr' = b.addr - n -> Addr
}
```

'' Operator

We can also predicate on the future future instance of a relation: this is often used in undo operations or reset of a predicate. Let's model the undo of the add operation.

```
assert delUndoesAdd{
    all b: Book, n: Name, a: Addr |
    no n.(b.addr) and add[b,n,a] and after del[b,n] implies
```

```
    b.addr = b.addr''
}
```

if in the first instant `n` does not belong to `b`, and `n` is added to `b`, and then, at the next instant (i.e., the second) `n` is removed from `b`, then address book `b` after 2 instants is the same as the initial one.

Combinations

> It is possible to combine those operators to obtain interesting predicates:
> **`eventually always P`** means that, from some instant in the future, predicate P will always be true.